Cryptocurrency Price Tracker

1. Project Title

Cryptocurrency Price Tracker using Python and Selenium

---

2. Submitted By

Name: Nithish Kumar V

---

3. Objective

The objective of this project is to develop a Python-based automation tool that extracts real-time cryptocurrency data such as coin name, current price, 24-hour change, and market capitalization from CoinMarketCap and stores it in a structured CSV format.

---

4. Project Description

The Cryptocurrency Price Tracker is a Selenium-based web scraping tool that automates browser interaction to collect live cryptocurrency data. Since CoinMarketCap loads data dynamically using JavaScript, Selenium is used to render the page and extract accurate information.

The script retrieves the top 10 cryptocurrencies and records their details along with a timestamp. The extracted data is then saved into a CSV file for further analysis, monitoring, or integration with dashboards.

---

5. Technologies Used

- **Python** – Programming language
- **Selenium** – For browser automation and dynamic scraping
- **pandas** – For data handling and CSV export
- **webdriver_manager** – For automatic ChromeDriver setup
- **time module** – For delays
- **datetime module** – For timestamp logging

---

6. Methodology

1. Initialize Chrome WebDriver using Selenium
2. Open the CoinMarketCap website
3. Wait for the cryptocurrency table to load
4. Scroll the page to ensure all data is rendered

5. Extract top 10 cryptocurrency details:
    - Name
    - Current Price
    - 24-hour Change
    - Market Capitalization
6. Add timestamp to each record
7. Store data in a list of dictionaries
8. Convert data into a pandas DataFrame
9. Export the data to a CSV file

---

7. Code Explanation

- Selenium WebDriver is used to control the browser
- Headless mode allows execution without opening a browser window
- WebDriverWait ensures the page is fully loaded before scraping
- XPath selectors are used to locate rows and columns in the table
- Data is extracted from specific column indices
- Each record is stored as a dictionary and appended to a list
- pandas is used to convert the list into a DataFrame
- The DataFrame is exported to a CSV file

---

8. Output

The output of the project is a CSV file named **crypto_prices.csv** containing the following columns:

- Name
- Current Price
- 24h Change
- Market Capitalization
- Timestamp

This file can be opened in Excel for analysis.

---

9. Advantages

- Provides real-time cryptocurrency data

- Automates data collection process
- Enables historical tracking using timestamps
- Can be used for data analysis and dashboards

---

10. Limitations

- Depends on website structure
- Requires internet connection
- Dynamic content may change frequently

---

11. Future Enhancements

- Track more than top 10 cryptocurrencies
- Add filtering options (e.g., highest gainers)
- Integrate with dashboards for visualization
- Schedule automatic data collection

---

12. Screenshots

Figure 1: Code Implementation

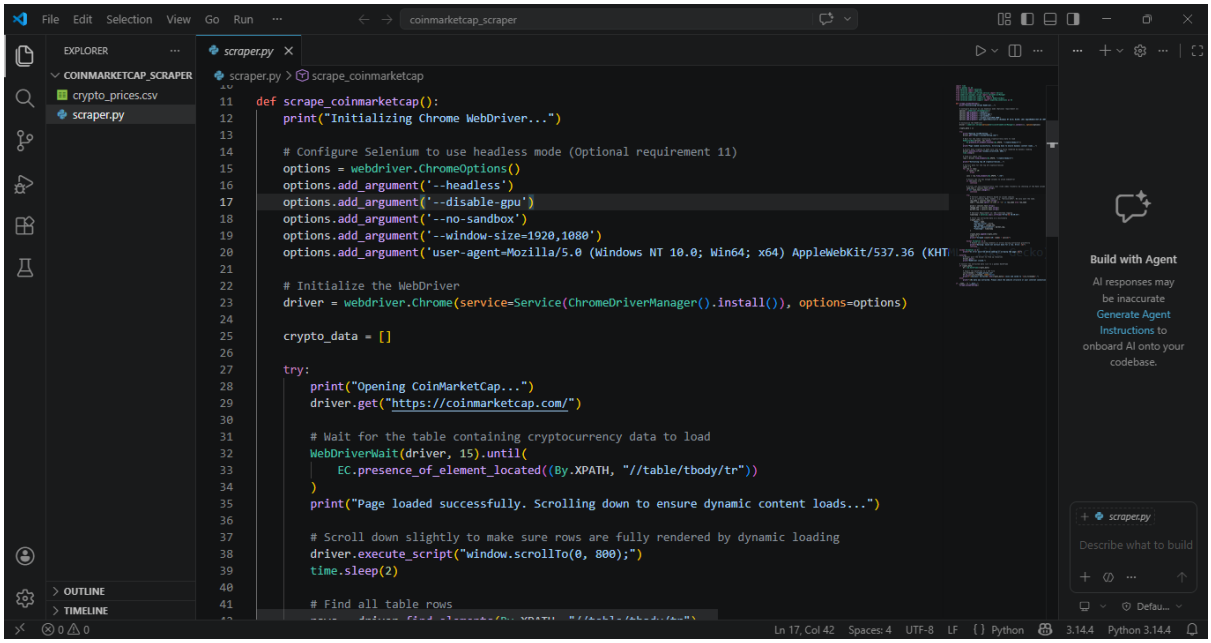


Figure 2: Program Execution

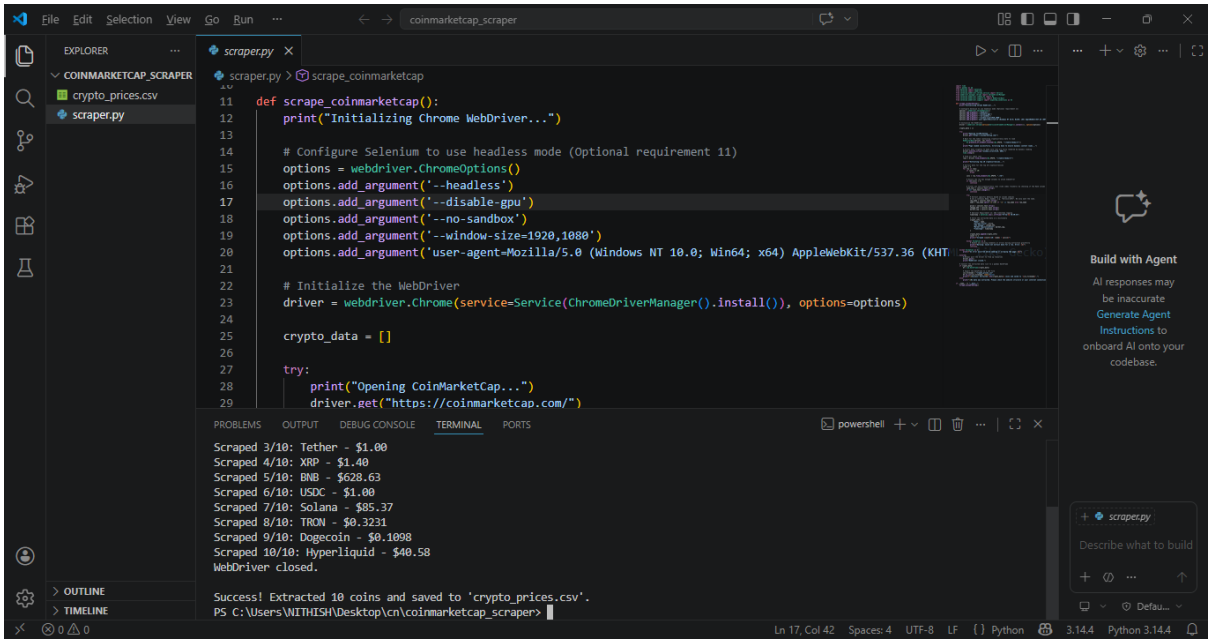
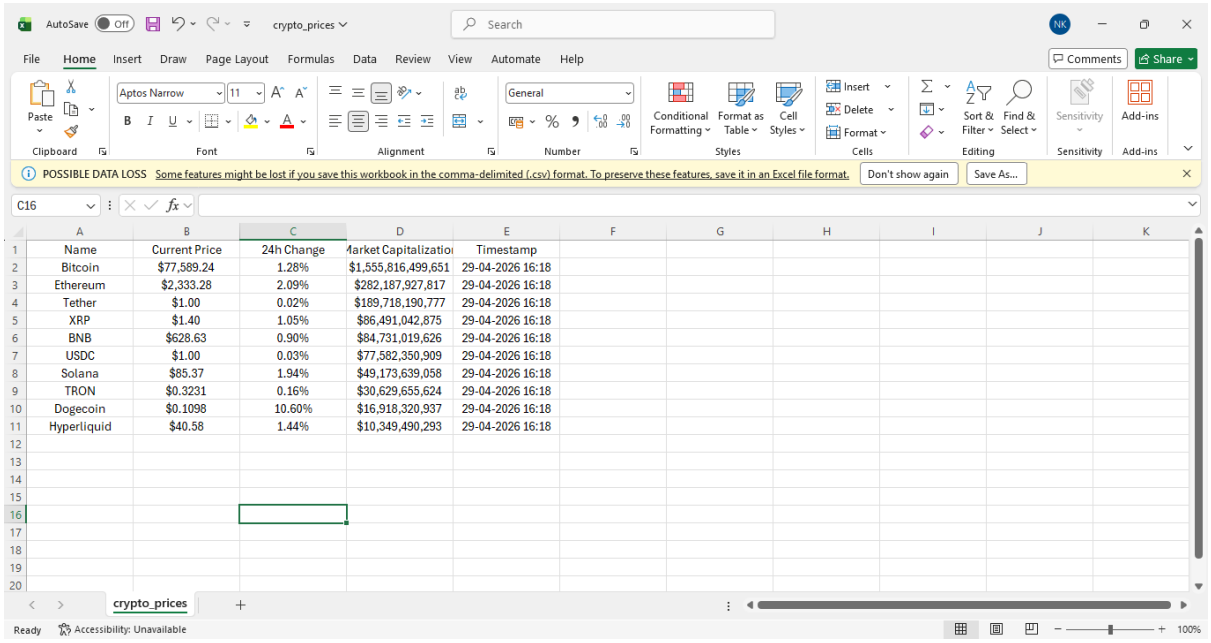


Figure 3: CSV Output



13. Conclusion

This project demonstrates how Selenium can be used to scrape real-time data from dynamic websites. It provides a practical understanding of web automation, data extraction, and storage. The project can be further extended for advanced analytics and financial applications.

14. Source Code

```
import time
import pandas as pd
from datetime import datetime
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def scrape_coinmarketcap():
    print("Initializing Chrome WebDriver...")

    # Configure Selenium to use headless mode (Optional requirement 11)
    options = webdriver.ChromeOptions()
    options.add_argument('--headless')
    options.add_argument('--disable-gpu')
    options.add_argument('--no-sandbox')
    options.add_argument('--window-size=1920,1080')
    options.add_argument('user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36')

    # Initialize the WebDriver
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)

    crypto_data = []

    try:
        print("Opening CoinMarketCap...")
        driver.get("https://coinmarketcap.com/")
```

```
    # Wait for the table containing cryptocurrency data to load
    WebDriverWait(driver, 15).until(
        EC.presence_of_element_located((By.XPATH, "//table/tbody/tr"))
    )
    print("Page loaded successfully. Scrolling down to ensure dynamic content loads...")

    # Scroll down slightly to make sure rows are fully rendered by dynamic loading
    driver.execute_script("window.scrollTo(0, 800);")
    time.sleep(2)

    # Find all table rows
    rows = driver.find_elements(By.XPATH, "//table/tbody/tr")

    print(f"Extracting top 10 cryptocurrencies...")

    # Extract data for the top 10 cryptocurrencies
    count = 0
    for row in rows:
        if count >= 10:
            break

        cols = row.find_elements(By.XPATH, "./td")

        # Ensure the row has enough columns to avoid IndexError
        if len(cols) < 8:
            continue

        # Filter out non-cryptocurrency rows (like index trackers) by checking if the Rank column is a number
        rank_text = cols[1].text.strip()
```

```
if not rank_text.isdigit():
    continue

try:
    # Extract specific details based on column indices
    # Col 2 contains Name, Symbol. e.g. "Bitcoin\nBTC". We only want the name.
    raw_name = cols[2].text.strip()
    name = raw_name.split('\n')[0] if '\n' in raw_name else raw_name

    price = cols[3].text.strip()
    change_24h = cols[5].text.strip()
    market_cap = cols[7].text.strip()

    # Optional Requirement 12: Add timestamp logging
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # Store the extracted data in a dictionary
    crypto_info = {
        "Name": name,
        "Current Price": price,
        "24h Change": change_24h,
        "Market Capitalization": market_cap,
        "Timestamp": timestamp
    }

    crypto_data.append(crypto_info)
    count += 1
    print(f"Scraped {count}/10: {name} - {price}")

except Exception as e:
    # Handle any missing elements or errors during extraction gracefully
```

```
            print(f"Warning: Could not extract data for a row. Error: {e}")
            continue

    except Exception as e:
        print(f"An error occurred while loading or scraping the page: {e}")

    finally:
        # Always quit the driver to free up resources
        driver.quit()
        print("WebDriver closed.")

    # Convert the extracted data list to a pandas DataFrame
    if crypto_data:
        df = pd.DataFrame(crypto_data)

        # Export the DataFrame to a CSV file
        csv_filename = "crypto_prices.csv"
        df.to_csv(csv_filename, index=False)
        print(f"\nSuccess! Extracted {len(crypto_data)} coins and saved to '{csv_filename}'.")
    else:
        print("\nNo data was extracted. Please check the website structure or your internet connection.")

if __name__ == "__main__":
    scrape_coinmarketcap()
```
